DATABASE SYSTEM
ECEG-4172

CHAPTER THREE:
Relational Data Model

Outline

- Structure of Relational Database
- Conceptual Data Model to Relational Model Mapping
- Dependencies
  - Functional Dependencies
  - Multivalued Dependencies
- Normal Forms and Normalization

Cont'd...

- Relational Data Model is an implementation (representational) model proposed by E.F. Codd in 1970.
- The model is an approach in a database design towards the Relational Database Management System (RDBMS).

Structure of Relational Database

- The main construct for representing data in the relational database is a two-dimensional **table** called a ***relation*****.**
  - The columns in the table are representing the ***attributes*** of the relationship, and
  - The rows (other than the heading row) represent ***tuples (records)*** of the relation.

Example:-"EMPLOYEES" relation

Employees					
EmpId	Name	BDate	Sub City	Kebele	Phone
E001	Alemu Girma	01/10/70	Bole	06	011-663-0712
E004	Kelem Belete	12/04/68	Gulele	03	011-227-2525

Fig. Typical Employee relation instance

- A relation in a relational model consists of:
  - The ***Relation schema***: - that describes the column heads for the table and
  - The ***Relation instance***: - that is the table with the set of tuples.

Cont'd...

- The set of relation schema forms schema for the relational database called **database schema (relational database schema)**.
- In relational model, the relation schema are described first. And the schema specifies
  - ***The relation's name***
  - ***Name for each attribute*** (field or column)
  - ***Domain*** of each attribute: - A domain is referred to in a relation schema by the domain name and has a set of associated values.

Example

- Employees (EmpId:sting, Name:string, BDate:date, SubCity:string, Kebel:integer, Phone:string)
- Projects (PrjId:integer, Name:string, SDate:date, DDate:date, CDate:date)
- Teams (Name:string, Descr:string)

Properties of Relations

- Rows (tuples) in a single relation are unique (that is; no two tuples are identical).
- Relations are set of tuples, not lists (that is; order of tuples in a relation is immaterial).
- Attributes are atomic.
- The values that appear in a column must be drawn from the domain associated with that column.
- The ***degree***, also called ***arity***, of a relation is the number of attributes in the relation.
- The relation names in a relational database are distinct.

Key Constraints

- A ***key constraint*** is a statement that a certain minimal subset of the attributes of a relation is a unique identifier for a tuple in the relation.
- A set of attributes that uniquely identifies a tuple according to a key constraint is called a ***candidate key*** for the relation; often abbreviated just as key.
- Key attributes in relational model are indicated by **underlying** the attributes in the relational.

Cont'd...

- Examples:
  - Employees (EmpId, Name, BDate, SubCity, Kebel, Phone)
  - Projects (PrjId, Name, SDate, DDate, CDate)
  - Teams (Name, Descr)

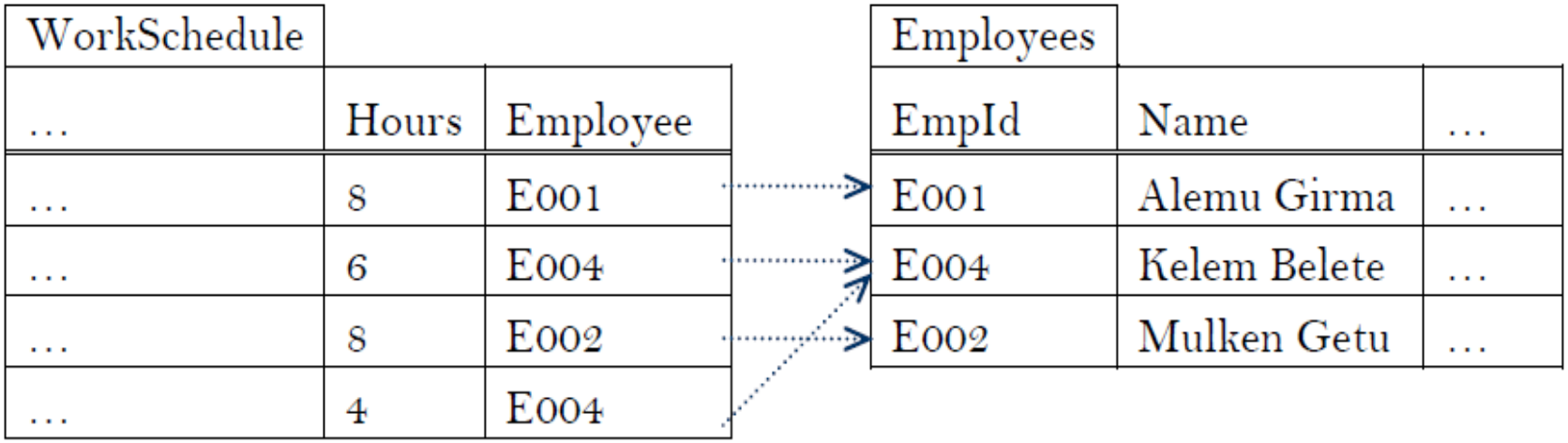
Foreign Key Constraints

- The most common integrity constraint involving two relations is a foreign key constraint.
  - It keeps data consistency when a data modification is done on a relation.
- The foreign key in the referencing relation requires a match to a primary key in the referenced relation. That is, there must be a compatible data type attribute in the referenced relation so as the referencing relation may make the referencing.

Cont'd...

- Example
  - Employees (**EmpId**, Name, BDate, SubCity, Kebel, Phone)
  - WorkSchedule (SDate, EDate, HoursPerDay, **Employee**)
- The "WorkSchedule" to refer to the "Employees" relation instance, it has an attribute "Employee" of the same type as the "EmpId" in the "Employees" relation which is a primary key.
- The foreign key constraint is implemented through the "Employee" attribute in the referencing relation "WorkSchedule".

Cont'd...



WorkSchedule

…	Hours	Employee
…	8	E001
…	6	E004
…	8	E002
…	4	E004

Employees

EmpId	Name	…
E001	Alemu Girma	…
E004	Kelem Belete	…
E002	Mulken Getu	…

(a) Referencing relation (b) Referenced relation

Fig. Foreign Constraint in Relational Model

Cont'd...

- **Note:-**
  - A single tuple can be referenced by zero or more tuples in the referencing relation, but a single tuple with a single foreign key attribute can only reference one tuple.
  - A foreign key could refer to the same relation.
  - A relational database consists of related relations through a foreign key.

Conceptual Data Model to Relational Model Mapping

- The second phase in database design is implementation design that
  - Transforms the conceptual data model into an internal model - schema such as a relational data model

From E/R Diagram to Relational Model

Entity Sets to Relations

- ***Strong entity sets*** in E/R model are mapped to relations in relational model with the same name and attributes.
- The primary keys assigned for the entity sets are also represented as keys in the relations.

Example

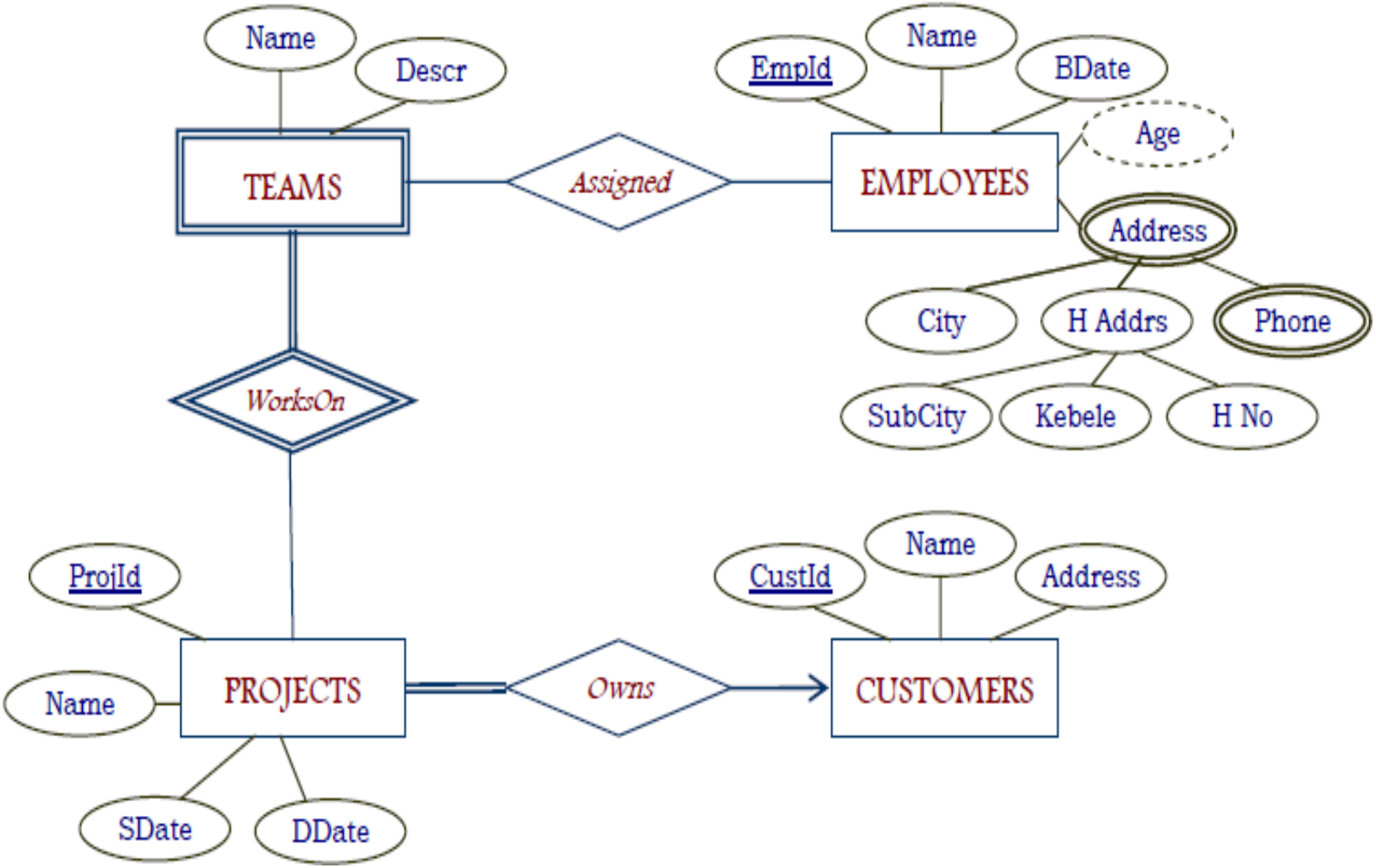


Fig . Partial E/R Model from Conceptual Data Model

... Example

- Then the relations from the strong entity sets having only simple and single valued attributes are as follows
  - Projects(ProjId, Name, SDate, DDate)
  - Customers(CustId, Name, Address)

Handling Weak Entity Sets

- Suppose
  - **W** is a ***weak entity set*** with attribute set **{a_1, a_2, a_3, … a_n}** and
  - ***identifying strong entity set*** **E**.
    - And let the ***primary key*** of E is the set **{b_1, b_2, … b_m},**
- Then the attributes of the relation for the weak entity set
  - Must include attributes for its complete key (including those belonging to the identifying strong entity set) and its own, non-key attributes.
  - That is, the set of attributes of the mapping relation is

    {a_1, a_2, a_3, … a_n} U {b_1, b_2, … b_m}.

Cont'd...

- The primary key for the weak entity set relation thus include:
  - The ***discriminator*** (attributes) of the weak entity set, and
  - The ***primary key*** of the identifying strong entity set.
- Example:- For the weak entity set (TEAMS) in figure above the corresponding relation is:
  - Teams(ProjId, Name, Descr)

Handling Composite and Multivalued Attributes

- **Composite attributes** from E/R model to a relational model can be represented by ***creating separate attributes for each*** of the components of the attributes (Note that the composite attribute is not mapped directly into a separate attribute).
- **Multivalued attributes** are handled by ***creating relations with the name of the attribute*** having attributes that corresponds to the components of the multivalued attribute and the primary key of the entity set or relationship set of which the attribute belongs. The primary key for the newly created relation consists of:
  - The primary key of the entity set or relationship set, and
  - The attribute or set of attributes from the multivalued attribute.

Cont'd...

- **Example**:- Consider the EMPLOYEES entity set in figure above the corresponding relations for the entity set are:
  - Employees(EmpId, Name, BDate, Age)
  - Addresses(City, SubCity, Kebele, HNo, Phone1, Phone2, EmpId)
- **REMARK**:-Note that; if the multivalued attribute has a fixed size of multiplicity (small size), it can be represented by separate attributes for each multiplicity. For example consider phone attribute above.

Relationship Sets to Relations

- Suppose entity set E with a primary key $\{a_{11}, a_{12}, a_{13}, \ldots a_{1n}\}$ is related to an entity set F with a primary key $\{a_{21}, a_{22}, a_{23}, \ldots a_{2m}\}$ through a relationship R. Let the relationship R has a descriptive attribute set $\{b_1, b_2, b_3, \ldots b_p\}$, then the relationship is represented by a relation whose attributes are:
  - The keys of the connected entity sets:

    $\{a_{11}, a_{12}, a_{13}, \ldots a_{1n}\} \cup \{a_{21}, a_{22}, a_{23}, \ldots a_{2m}\}$, and
  - Attributes of the relationship itself:

    $\{b_1, b_2, b_3, \ldots b_p\}$.

Cont'd...

- The union of the primary keys of the related entity sets forms super key for the relationship relation.
- If the relationship is many-to-many the super key also becomes a primary key for the relation, otherwise the primary key from the many side becomes the primary key for the relation.
- Example:- From figure above the corresponding relations for the relationship sets are:
  - Assigned(EmpId, ProjId, TeamName)
  - Owns(ProjId, CustId)

Cont'd...

- **NOTE**: Supporting relationships (for example WorksOn) need not be transformed to relations if their purpose is solely for identifying a weak entity set by passing on the identifying strong entity set's primary key to the weak entity set; otherwise they will introduce redundancy.

Cont'd...

- Suppose entity set E and F are related through a **many-to-one** relationship **R from E to F**, then it is possible to join the relations for E and R that come out of this E/R model into a single relation S with a schema consisting of:
  - All attributes of the entity set E,
  - The keys attributes of the entity set F, and
  - All Attributes of the relationship R.

Cont'd...

- If the participation of E into R is total it is also possible to include all attributes of F in the relation S and have one single relation S in place of the three relations E, F and R.
- The primary key for S would the primary key of E.

Cont'd...

- Example:-Consider the entity sets "PROJECTS" and "CUSTOMERS" and the corresponding relationship "Owns", then we can have:
  - Projects(ProjId, Name, SDate, DDate, CustId) and Customers(CustId, Name, Address), or
  - Projects(ProjId, Name, SDate, DDate, CustId, Name, Address

Representation of Generalization and Specialization (Subclass)

- Hierarchical structure (Specialization and Generalization or Inheritance) in relation model can be represented in three different ways:
  1. ***E/R Style***
  2. ***Use of Nulls***
  3. ***Object-Oriented Approach***

Cont'd...

1. ***E/R Style:*** One relation for each lower-level entity set and the higher-level entity set.
   - Every relation of the lower-level entity set will include:
     - Key attribute(s) of the higher-level entity set which forms the primary key of the entity set, and
     - Attributes of that lower-level entity set.
   - For total and disjoint generalization the higher-level entity set may not be mapped into a relation instead all its attributes are passed to all immediate lower-level entity sets relations.

Cont'd...

2. ***Use of Nulls :*** One relation having a large set of attributes of all the lower-level entity sets and higher-level entity set; entities have NULL in attributes that don't belong to them.
   - Involves large number of NULL values for disjoint generalization.
3. ***Object-Oriented Approach:*** One relation per subset of subclasses, with all relevant attributes including:
   - Attributes of the higher-level entity set, and
   - Attributes of that lower-level entity set.
   - The primary key of the higher-level entity set becomes the primary key of each relation.

Cont'd...

- Example:- Consider the entity sets "EMPLOYEES" and its lower-level entity sets, then
  - FullTimeEmployees(EmpId, Salary, Saving, Allowance)
  - PartTimeEmployees(EmpId, HourlyPay, ContractPeriod)

From ODL Model to Relational Model

Cont'd...

- ODL model to relational model mapping is similar to the mapping process of E/R model to relational.

ODL Class to Relations

- ODL classes are directly mapped to relations in relational model with their attribute as in the case of E/R.
  - Unlike E/R entity sets ODL classes optionally define key, in such events it is necessary to add new attribute as a primary key for the relation.
- ***Non-atomic attributes*** in ODL classes are represented by simply expanding the structure definition and making one attribute for each field of the structure.

- Example:- Consider the “EMPLOEES” class partial declaration in chapter 2, the corresponding relational model is:

```
class Employee (extent Employees key EmpId, NationalId, (Name, BDate)) {          attribute string empId;
attribute string name;
attribute integer age;
attribute enum Gender {Male, Female} gender;
attribute struct Address {string city, string hAddr, string phone} address;
:
};
```

- Then the corresponding relational model is
  - Employees(<u>EmpId</u>, Name, Age, Gender, City, HAddr, Phone)

Cont'd...

- Set-valued attributes:
  - Recall that attributes in ODL class can be constructed from the type constructors Set, Bag, List and Array.
  - Such cases can be handled by three different methods:

1.By making one tuple for each value.

Cont'd...

- Example:- Consider the "EMPLOEES" class having a set of addresses:

```
class Employee (extent Employees key EmpId, NationalId, (Name, BDate)) {
        .
        .
attribute struct set Address {string city, string hAddr, string phone}
address;
:
};
```

- Then the corresponding relational model is
  - Employees(EmpId, Name, Age, Gender, City, HAddr, Phone)

Cont'd...

- The relational schema may have the following instance

Employees					
EmpId	Name	BDate	Sub City	Kebele	Phone
E001	Alemu Girma	01/10/70	Bole	06	011-663-0712
E004	Kelem Belete	12/04/68	Gulele	03	011-227-2525

Cont'd...

2. By separating out each set-valued attribute into a new relation and establishing a many-to-many relationship.
3. By having multiple attributes sets for each set-valued attribute. This is applicable only if the type constructor is fixed size array.

ODL Relationships to Relations

- Recall that ODL relationships are represented in pair as inverse relationship, hence in representing ODL relationships only one of the declarations are used.
- Similar to E/R relationships, ODL relationships can also be represented by a relation having the primary keys of the related classes as attributes.

- **Example**:- Consider the relationship between "EMPLOYEES" class and the "TEAMS" class.

```
class Employee {
        relationship Set <Team> assigned
                inverse Team::formed;
};
class Team {
        relationship Set <Employee> formed;
                inverse Employee::assigned;
};
```

- Then the corresponding relational models are
  - Employees(<u>EmpId</u>, Name, Age, Gender, City, HAddr, Phone)
  - Teams(<u>TeamId</u>, Name, Descr)
  - Assigned(<u>EmpId</u>, <u>TeamId</u>)

Cont'd...

- For many-to-one relationship the relationship relation can be combined with the many side relation as in the case of E/R relationship.

- **Example**:- Consider the one-to-many relationship between “PROJECTS” and “CUSTOMERS”

```
class Project {
        relationship <Customer> ownedBy
                inverse Customers::owns;
};
class Customer {
        relationship Set <Project> owns;
                inverse Projects::ownedBy;
};
```

- Then the corresponding relational models are
  - Projects(ProjId, Name, SDate, DDate, CustId)
  - Customers(CustId, Name, Address)

Dependencies

- In a database design, the two most common pitfalls that result in bad designing are:
  - Repetition of information, and
  - Inability to present certain information (Loss of information).

Functional Dependencies

- Functional dependency is a kind of constraint that helps to remove redundancy in relational database design.
- ***Definition***:
  - Functional dependency denoted by **X** $\rightarrow$**A** is an assertion about a relation ***R*** that whenever two tuples of ***R*** agree on all the attributes of ***X***, then they must also agree on the attribute ***A***. We say that "***X*** $\rightarrow$ ***A holds in*** *R*" or "***X functional determines A***"
  - Note that in the notation $X \rightarrow A$; X represent sets of attributes and *A* represent single attribute. That is $A_1A_2A_3 \ldots A_n \rightarrow B$
- The functional dependency is a generalization of the notion of superkey.

Cont'd…

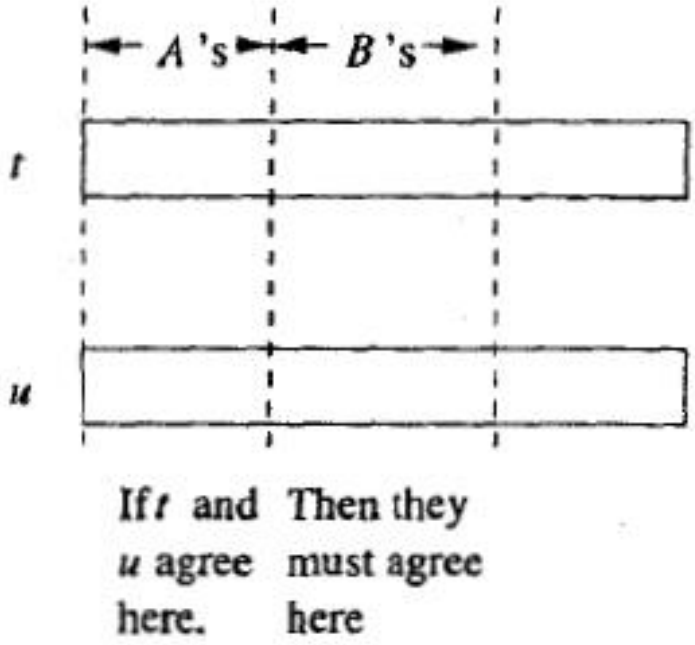


Figure 3.1: The effect of a functional dependency on two tuples.

Example:

- Consider the Teams relation:
  - Teams(PrjId, Name, Descr), then
  - ***PrjId, Name → Descr***
- For the Employees relation:
  - Employees(EmpId, NationalId, Name, BDate, Age, Gender, City, HAddr, Phone)
  - ***EmpId → Name; EmpId → Age; Name BDate → Gender***
- A functional dependency $A_1A_2A_3 \ldots A_n \rightarrow B$ is said to be ***trivial dependency*** if B is an element or of {A1, A2, A3…An}.

Rules of Functional Dependency

- **Combining Rule:**
  - The functional dependencies:

$$A_1A_2A_3\ldots A_n \rightarrow B_1$$
$$A_1A_2A_3\ldots A_n \rightarrow B_2$$
$$A_1A_2A_3\ldots A_n \rightarrow B_3$$
$$.$$
$$.$$
$$A_1A_2A_3\ldots A_n \rightarrow Bm$$

Can be written as

$$A_1A_2A_3\ldots A_n \rightarrow B_1\ B_2B_3\ldots B_m$$

Cont'd...

- **Splitting Rule:**
  - The functional dependencies:

$$A_1A_2A_3\ldots A_n \rightarrow B_1\ B_2B_3\ldots B_m$$

Can be written as

$$A_1A_2A_3\ldots A_n \rightarrow B_i \text{ for } i=1,2,3\ldots m$$

Cont'd...

➢Closure of Attributes

- Suppose {A1, A2, …An} is a set of attributes and S is a set of functional dependencies in relation R.
- The closure of the set {A1, A2, …An} under the functional dependency set S is the set of attributes B that are functionally determined from the set S.
- That is; $A_1A_2A_3\ldots A_n \rightarrow B$ follows from the set S.
- The closure set of attributes A1, A2, …An is denoted by $\{A1, A2, \ldots An\}^+$
- The closure set of attributes can be determined by repeatedly applying the following three rules known as ***Armstrong's Axioms***.

Cont'd...

- **Reflexivity Rule:**
  - If α is set of attributes and $\beta \subseteq \alpha$ then, $\alpha \rightarrow \beta$ holds.
- **Augmentation Rule:**
  - If $\alpha \rightarrow \beta$ holds and γ is set of attributes, then $\gamma\alpha \rightarrow \gamma\beta$ holds.
- **Transitivity Rule:**
  - If $\alpha \rightarrow \beta$ holds and $\beta \rightarrow \gamma$ holds, then $\alpha \rightarrow \gamma$ holds.

Cont'd...

- Algorithm for computing the closure of X, X^+ is given below.
  1. Let X be a set of attributes that eventually will become the closure. First, we initialize X to be X.
  2. Now, we repeatedly search for some functional dependency $B_1B_2 \ldots B_n \rightarrow C$ Such that all of B_1, B_2...B_m are in the set of attributes X but C is not. We then add C to the set X.
  3. Repeat step 2 as many times as necessary until no more attributes can be added to X.
  4. The set X, after no more attributes can be added to it, is the closure set X^+.

Example:

- Consider a relation with attributes A, B, C, D, E, and F. Suppose that this relation has the functional dependencies $AB \rightarrow C$, $BC \rightarrow AD$, $D \rightarrow E$, and $CF \rightarrow B$. What is the closure of {A, B}, that is {A, B}?
- **Solution**:
  - X = {A, B}
  - From the function dependency $AB \rightarrow C$, we add C to X that is X = {A, B, C}
  - Similarly; $BC \rightarrow AD \Rightarrow$ X = {A, B, C, D}
  - $D \rightarrow E \Rightarrow$X = {A, B, C, D, E}
  - No more changes in X are possible. Thus $\{A, B\}^+ = \{A, B, C, D, E\}$
  - From the closure set it is to follow that $AB \rightarrow D$

Exercise: Test whether D → A flows from the functional dependency set?

- To test for D → A, first determine the closure set of {D}
- X = {D}
- From the function dependency D → E, we add E to X that is X = {D, E}
- No more changes in X are possible. Thus $\{D\}^+ = \{D, E\}$
- From the closure set D → A does not hold.

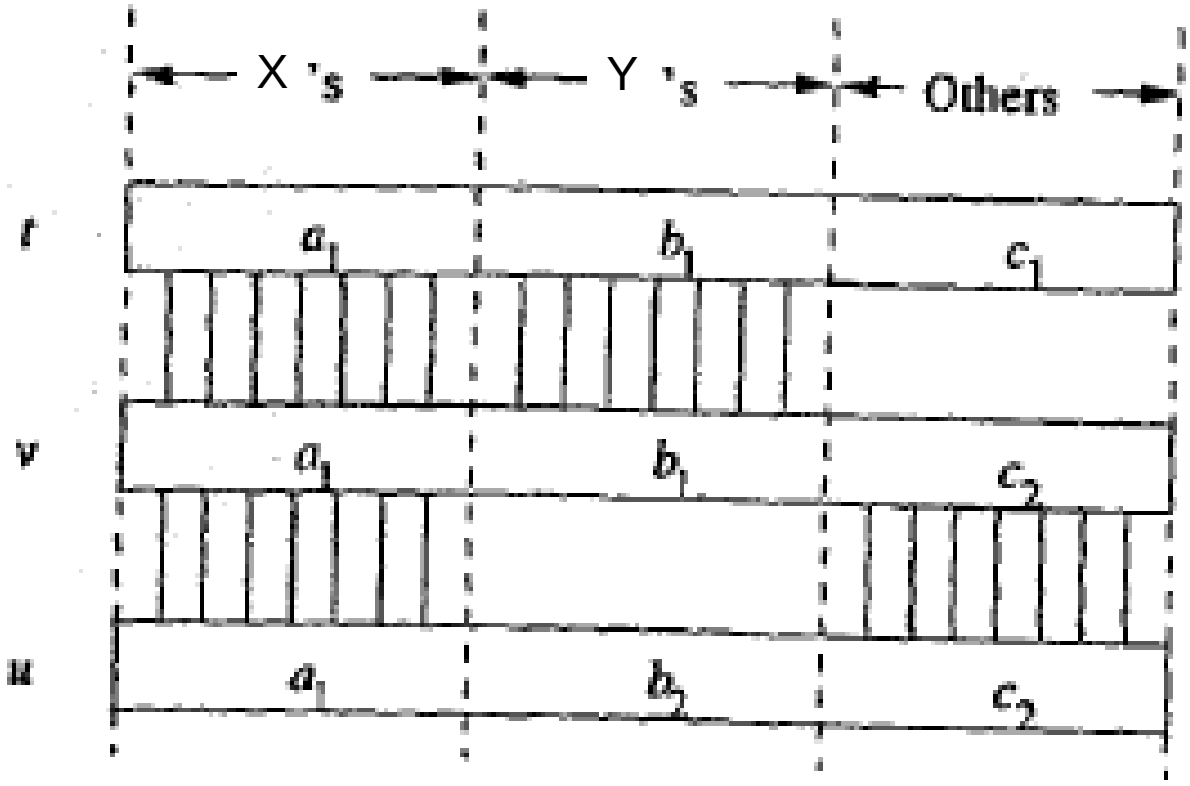
Multivalued Dependencies

- Multivalued dependency for a relation R, is defined as a constraint when the values of one set of attributes is fixed, then the values in certain other attributes are independent of values of all the other attributes in R.

Cont'd...

- That is; for a multivalued dependency X →→Y in R where X and Y are subsets of the set of attributes in R, if ***t*** and ***u*** are tuples in the relational instance ***r*** for the schema R that agree on all the X's, then there exist a third tuple ***v*** that agrees:
  1. with both t and u on X's,
  2. with ***t*** on Y's, and
  3. with ***u*** on all attributes of R that are not among X's or Y's (R – (X U Y)).

Cont'd...



Rules of Multivalued Dependency

- Multivalued dependency is a generalization for the functional dependency. That is;
  - If $\alpha \rightarrow \beta$ holds, then $\alpha \rightarrow\rightarrow \beta$ also holds.
- All the rules except the splitting rule for the functional dependency are also applicable for a multivalued dependency.

Cont'd...

Complementation Rule:

- One additional rule in a multivalued dependency is the complementation rule.
  - The rule states that if $X \rightarrow\rightarrow Y$ holds then $X \rightarrow\rightarrow (R - (X \cup Y))$, where R is a set of attributes for the relational schema R.

Normal Forms and Normalization

- In relational databases, ***normalization*** is a process that helps to
  - eliminates redundancy,
  - organizes data efficiently,
  - reduces the potential for anomalies during data operations, and
  - improves data consistency.
- The formal classifications used for quantifying "how normalized" a relational database are called ***normal forms*** (abbreviated as ***NF***).

Cont'd...

- Redundancy in a database design results in data anomalies classified as:
  - Insertion Anomalies
  - Deletion Anomalies
  - Modification Anomalies

Example:

- Consider a relation schemas for Employees and Teams in a single realtion as follows
  - Emp_Teams(EmpId, Name, BDate, Gender, TeamId, Project, TeamName)
- It can easily be noted that there is redundancy of data in the "Emp_Teams" relation for the Teams detail of the Employees.
- Consider the following instance for the relations

...Example:

Employees						
EmpId	Name	BDate	Gender	TeamId	Project	TeamName
E001	Alemu Girma	01/10/70	M	1	1	Programmer
E001	Alemu Girma	01/10/70	M	3	2	Programmer
E004	Kelem Belete	12/04/68	M	2	1	Tester
E005	Mulu Tasew	10/05/69	F	3	2	Programmer
E008	Belachew K	02/11/62	M	1	1	Programmer
E003	Almaz B	05/06/65	F	5	3	Programmer
E005	Mulu Tasew	10/05/69	F	2	1	Tester

…Example:

- **Insertion Anomalies**: Suppose we want to insert a new employee that works in project 1 as a programmer, then the corresponding fields for the Team detail has to be entered correctly. If data is entered incorrectly the consistency will be violated.
- **Deletion Anomalies**: Suppose E003 is to be removed from the employees list, then Team information of TeamId 5 will also be removed and vice versa.
- **Modification Anomalies**: During data update the consistency may also be violated as in the case of insertion.

Normal Forms

- Normalization procedure provides:
  - A framework for analysing relation schemas based on functional and multivalued dependencies.
  - A series of normal form test that can be carried out on individual relation schemas so that the relational database can be normalized to any degree.

First Normal Form (1NF)

- A relation (table) R is in 1NF
  - if and only if all underlying domains of attributes contain only atomic (simple, indivisible) values.
- 1NF allows removal of multivalued attributes, composite attributes and their combination in the relational schema.
- Normalization (Decomposition)
  - Form new relation for each non-atomic attribute or nested relation.

1st Normal Form

First Normal Form is violated if:

- The relation has no identifiable primary key.
- Any attempt has been made to store a multi-valued fact in a tuple.

1NF Example – Schema 1 (correct)

Programs Table

EMPID	LANGUAGE
23	COBOL
23	JAVA
23	SQL
31	SQL
32	JAVA
32	SQL
32	VB
32	COBOL
36	VB
36	SQL
36	JAVA
37	COBOL
37	SQL

Employees Table

EMPID	LNAME	FNAME	SEX	DEPT	PHONE	SALARY
23	Jones	Mark	M	ITR	555-1087	45000
25	Smith	Sara	F	FINC	555-2222	55000
26	Billings	David	M	ACTG	555-4356	42000
31	Dance	Ivanna	F	ACTG	444-4887	60000
32	Jones	Mary	F	ITR	555-8745	70000
35	Barker	Bob	M	ACTG	555-6565	44000
36	Woods	Robin	M	ITR	555-9812	90000
37	Jones	Mary	F	FINC	555-1234	56000

Languages Table

NAME	FULLNAME
COBOL	COmmon Business Oriented Language
JAVA	JAVA
SQL	Structured Query Language
VB	Visual Basic

1NF Example – Schema 2 (incorrect)

Employees Table

EMPID	LNAME	FNAME	SEX	DEPT	PHONE	SALARY	LANGUAGES
23	Jones	Mark	M	ITR	555-1087	45000	COBOL, JAVA, SQL
25	Smith	Sara	F	FINC	555-2222	55000	
26	Billings	David	M	ACTG	555-4356	42000	
31	Dance	Ivanna	F	ACTG	444-4887	60000	SQL
32	Jones	Mary	F	ITR	555-8745	70000	JAVA, SQL, VB, COBOL
35	Barker	Bob	M	ACTG	555-6565	44000	
36	Woods	Robin	M	ITR	555-9812	90000	VB, SQL, JAVA
37	Jones	Mary	F	FINC	555-1234	56000	COBOL, SQL

Languages Table

NAME	FULLNAME
COBOL	COmmon Business Oriented Language
JAVA	JAVA
SQL	Structured Query Language
VB	Visual Basic

1NF Example – Schema 3 (incorrect)

Employees Table

EMPID	LNAME	FNAME	SEX	DEPT	PHONE	SALARY	LANG1	LANG2	LANG3	LANG4
23	Jones	Mark	M	ITR	555-1087	45000	COBOL	JAVA	SQL	
25	Smith	Sara	F	FINC	555-2222	55000				
26	Billings	David	M	ACTG	555-4356	42000				
31	Dance	Ivanna	F	ACTG	444-4887	60000	SQL			
32	Jones	Mary	F	ITR	555-8745	70000	JAVA	SQL	VB	COBOL
35	Barker	Bob	M	ACTG	555-6565	44000				
36	Woods	Robin	M	ITR	555-9812	90000	VB	SQL	JAVA	
37	Jones	Mary	F	FINC	555-1234	56000	COBOL	SQL		

Languages Table

NAME	FULLNAME
COBOL	COmmon Business Oriented Language
JAVA	JAVA
SQL	Structured Query Language
VB	Visual Basic

1NF Example – Schema 4 (incorrect)

Employees Table

EMPID	LNAME	FNAME	SEX	DEPT	PHONE	SALARY	COBOL	JAVA	SQL	VB
23	Jones	Mark	M	ITR	555-1087	45000	T	T	T	F
25	Smith	Sara	F	FINC	555-2222	55000	F	F	F	F
26	Billings	David	M	ACTG	555-4356	42000	F	F	F	F
31	Dance	Ivanna	F	ACTG	444-4887	60000	F	F	T	F
32	Jones	Mary	F	ITR	555-8745	70000	T	T	T	T
35	Barker	Bob	M	ACTG	555-6565	44000	F	F	F	F
36	Woods	Robin	M	ITR	555-9812	90000	F	T	T	T
37	Jones	Mary	F	FINC	555-1234	56000	T	F	T	F

Languages Table

NAME	FULLNAME
COBOL	COmmon Business Oriented Language
JAVA	JAVA
SQL	Structured Query Language
VB	Visual Basic

Second Normal Form (2NF)

- A relation schema R is in 2NF
  - if it is in 1NF and
  - every non-prime attribute A in R is fully functionally dependent on the primary key. (i.e. not partially dependent on candidate key).
- Functional dependency X →Y is said to be *fully functionally* dependent if removal of any attribute from X result in for the dependency not hold.
- NOTE: Mostly relational schemas that are mapped carefully from E/R and ODL model are in 2NF.

...Second Normal Form (2NF)

- **Normalization (Decomposition)**
  - Decompose and set up a new relation for each partial key with its dependent attribute(s). Make sure to keep relation with the original primary key and any attributes that are fully functionally dependent on it.

…Second Normal Form (2NF)

- **Example**: Consider a relation schemas for Employees and Teams in a single realtion as follows
  - Emp_Teams(EmpId, Name, BDate, Gender, TeamId, Project, TeamName)
  - EmpId → Name, BDate, Gender
  - TeamId → Project, TeamName
- Then upon decomposition we will have
  - Employees(EmpId, Name, BDate, Gender)
  - Teams(TeamId, Project, TeamName)
  - Emp_Teams(EmpId, TeamId)

2nd Normal Form

Second Normal Form is violated if:

- First Normal Form is violated
- If there exists a non-key field(s) which is functionally dependent on a partial key.

partial key $\longrightarrow$ non-key

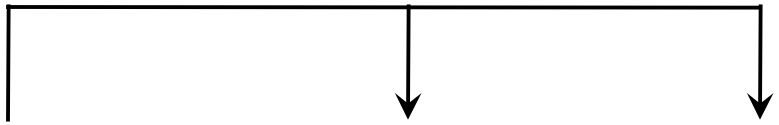
2NF Example – Violation

Transactions Table

JENO	LINENO	DATE	DESCRIPTION	ACCTNO	ACCTNAME	AMOUNT
1	1	02-JAN-2003	Owner investment	100	Cash	20,000
1	2	02-JAN-2003	Owner investment	310	Smith-Capita	(20,000)
2	1	03-JAN-2003	Borrowed money	100	Cash	30,000
2	2	03-JAN-2003	Borrowed money	220	Notes Payabl	(30,000)
3	1	03-JAN-2003	Purchased Supplies	120	Supplies	5,000
3	2	03-JAN-2003	Purchased Supplies	100	Cash	(1,000)
3	3	03-JAN-2003	Purchased Supplies	220	Notes Payabl	(4,000)

Is there a non-key field which is functional dependent on a partial key?

2NF Example – Violation
FDs that indicate violation of 2NF



JENO	LINENO	DATE	DESCRIPTION	ACCTNO	ACCTNAME	AMOUNT
1	1	02-JAN-2003	Owner investment	100	Cash	20,000
1	2	02-JAN-2003	Owner investment	310	Smith-Capita	(20,000)
2	1	03-JAN-2003	Borrowed money	100	Cash	30,000
2	2	03-JAN-2003	Borrowed money	220	Notes Payabl	(30,000)
3	1	03-JAN-2003	Purchased Supplies	120	Supplies	5,000
3	2	03-JAN-2003	Purchased Supplies	100	Cash	(1,000)
3	3	03-JAN-2003	Purchased Supplies	220	Notes Payabl	(4,000)

2NF Example – Corrected

Journal_Entry Table

JENO	DATE	DESCRIPTION
1	02-JAN-2003	Owner investment
2	03-JAN-2003	Borrowed money
3	03-JAN-2003	Purchased Supplies

Transactions Table

JENO	LINENO	ACCTNO	ACCTNAME	AMOUNT
1	1	100	Cash	20,000
1	2	310	Smith-Capita	(20,000)
2	1	100	Cash	30,000
2	2	220	Notes Payabl	(30,000)
3	1	120	Supplies	5,000
3	2	100	Cash	(1,000)
3	3	220	Notes Payabl	(4,000)

Third Normal Form (3NF)

- 3NF for a relation schema R requires that
  - the R be in 2NF, and
  - that there would be no nonprime attribute of R that has transitive dependencies on the primary key.
- In summary, all non-key attributes are mutually independent. Thus, any relation in which all the attributes are prime attributes (part of some key) is guaranteed to be in at least 3NF.

…Third Normal Form (3NF)

- That is; if X → Y is non-trivial functional dependency in R, then
  - X is superkey for schema R, or
  - Attribute Y is a member of a candidate key (prime attribute).
- **Normalization (Decomposition)**
  - Decompose and set up a relation that includes the non-key attribute(s) that functionally determine(s) other non-key attributes.

…Third Normal Form (3NF)

Boyce-Codd Normal Form (BCNF)

- BCNF requires that there will be no non-trivial functional dependencies of attributes on something other than a superset of a candidate key (called a superkey). At this stage, all attributes are dependent on a key, a whole key and nothing but a key (excluding trivial dependencies).
- A table is said to be in the BCNF if and only if it is in the 3NF and every non-trivial, left-irreducible functional dependency has a candidate key as its determinant. In more informal terms, a table is in BCNF if it is in 3NF and the only determinants are the candidate keys.

...Third Normal Form (3NF)

- That is; if X → Y is non-trivial functional dependency in R, then
  - X is superkey for schema R.
- Note that major goals of database design with functional dependencies are:
  - BCNF,
  - Lossless join, and
  - Dependency preservation;
- However; in certain situations it is needed to compromise BCNF need with 3NF to preserve dependency.

. . . Third Normal Form (3NF)

- **Example:**
  - A relation that is in 3NF form but not in BCNF:
  - R(A, B, C, D) and F = {AB →CD, BC →AD, A →C}
    - AB and BC are candidate keys, thus
    - A →C will not violet 3NF where as it violets BCNF since A is not superkey.
  - A relation that is in 3NF form and in BCNF:
    - R(A,B) is guaranteed to be in BCNF since its only possible functional dependencies are A →B, B →A and/or the trivial AB →AB.

…Third Normal Form (3NF)

- **Example**: Consider the Project relation from the E/R model
  - Projects(<u>ProjId</u>, Name, SDate, DDate, CustId, Name, Address)
    - ProjId → Name, SDate, DDate, CustId, Name, Address
    - CustId → Name, Address
  - Then upon decomposition we will have
    - Projects(<u>ProjId</u>, Name, SDate, DDate, CustId)
    - Customers(<u>CustId</u>, Name, Address)

3rd Normal Form

Third Normal Form is violated if:

- Second Normal Form is violated
- If there exists a non-key field(s) which is functionally dependent on another non-key field(s).

non-key ⟶ non-key

Note: A candidate key is not a non-key field.

3NF Example – Violation

Journal_Entry Table

JENO	DATE	DESCRIPTION
1	02-JAN-2003	Owner investment
2	03-JAN-2003	Borrowed money
3	03-JAN-2003	Purchased Supplies

Are there any non-key fields which functional determine another non-key field?

Are there any redundant facts?

Transactions Table

JENO	LINENO	ACCTNO	ACCTNAME	AMOUNT
1	1	100	Cash	20,000
1	2	310	Smith-Capita	(20,000)
2	1	100	Cash	30,000
2	2	220	Notes Payabl	(30,000)
3	1	120	Supplies	5,000
3	2	100	Cash	(1,000)
3	3	220	Notes Payabl	(4,000)

3NF Example – Violation
FD that indicates violation of 3NF

Anomalies if not corrected:

- **update** (if name of account 100 changes it must be changed in multiple places risking inconsistancy)
- **deletion** (can't delete JE#3 and its transactions without losing information about account 120)
- **insertion** (can't set up a new account, Jones-capital, for a new partner unless we first have a transaction involving that account.

Journal_Entry Table

JENO	DATE	DESCRIPTION
1	02-JAN-2003	Owner investment
2	03-JAN-2003	Borrowed money
3	03-JAN-2003	Purchased Supplies



JENO	LINENO	ACCTNO	ACCTNAME	AMOUNT
1	1	100	Cash	20,000
1	2	310	Smith-Capita	(20,000)
2	1	100	Cash	30,000
2	2	220	Notes Payabl	(30,000)
3	1	120	Supplies	5,000
3	2	100	Cash	(1,000)
3	3	220	Notes Payabl	(4,000)

3NF Example – Corrected

Accounts Table

ACCTNO	ACCTNAME
100	Cash
120	Supplies
220	Notes Payable
310	Smith-Capital

Journal_Entry Table

JENO	DATE	DESCRIPTION
1	02-JAN-2003	Owner investment
2	03-JAN-2003	Borrowed money
3	03-JAN-2003	Purchased Supplies

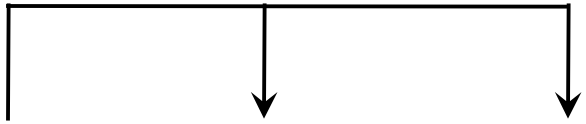
Transactions Table

JENO	LINENO	ACCTNO	AMOUNT
1	1	100	20,000
1	2	310	(20,000)
2	1	100	30,000
2	2	220	(30,000)
3	1	120	5,000
3	2	100	(1,000)
3	3	220	(4,000)

3NF Example – Corrected
Final Dependencies



ACCTNO	ACCTNAME
100	Cash
120	Supplies
220	Notes Payable
310	Smith-Capital



JENO	DATE	DESCRIPTION
1	02-JAN-2003	Owner investment
2	03-JAN-2003	Borrowed money
3	03-JAN-2003	Purchased Supplies



JENO	LINENO	ACCTNO	AMOUNT
1	1	100	20,000
1	2	310	(20,000)
2	1	100	30,000
2	2	220	(30,000)
3	1	120	5,000
3	2	100	(1,000)
3	3	220	(4,000)

All non-key fields are FD on the PK and only the PK.

BCNF Normal Form

Boyce-Codd Normal Form is violated if:

- Third Normal Form is violated
- If there exists a partial key which is functionally dependent on a non-key field(s).

non-key $\longrightarrow$ partial-key

BCNF Example Semantics

- A student can have more than one major
- A student has a different advisor for each major.
- Each advisor advises for only one major.

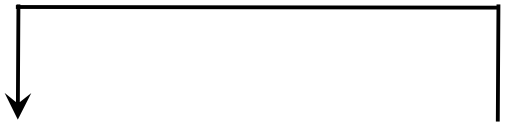
BCNF Example – Violation

Student_Majors Table

SID	MAJOR	ADVISOR
1	PHYSICS	EINSTEIN
1	BIOLOGY	LIVINGSTON
2	PHYSICS	BOHR
2	COMPUTER SCIENCE	CODD
3	PHYSICS	EINSTEIN
4	BIOLOGY	LIVINGSTON
4	ACCOUNTING	PACIOLI
5	PHYSICS	EINSTEIN
6	PHYSICS	BOHR
6	BIOLOGY	DARWIN
7	COMPUTER SCIENCE	CODD
7	BIOLOGY	DARWIN

Does this relation violate third normal form?
Are there any redundant facts?

BCNF Example – Violation
FD that violates BCNF



It is important that you convince yourself that major does not FD advisor.

SID	MAJOR	ADVISOR
1	PHYSICS	EINSTEIN
1	BIOLOGY	LIVINGSTON
2	PHYSICS	BOHR
2	COMPUTER SCIENCE	CODD
3	PHYSICS	EINSTEIN
4	BIOLOGY	LIVINGSTON
4	ACCOUNTING	PACIOLI
5	PHYSICS	EINSTEIN
6	PHYSICS	BOHR
6	BIOLOGY	DARWIN
7	COMPUTER SCIENCE	CODD
7	BIOLOGY	DARWIN

BCNF Example – Corrected

Advisors Table

ADVISOR	MAJOR
BOHR	PHYSICS
CODD	COMPUTER SCIENCE
DARWIN	BIOLOGY
EINSTEIN	PHYSICS
LIVINGSTON	BIOLOGY
PACIOLI	ACCOUNTING

Note that the if the original key, counter-intuitively, in schema 1 had been defined as SID & ADVISOR this would have been a 2NF violation.

Student_Advisors Table

SID	ADVISOR
1	EINSTEIN
1	LIVINGSTON
2	BOHR
2	CODD
3	EINSTEIN
4	LIVINGSTON
4	PACIOLI
5	EINSTEIN
6	BOHR
6	DARWIN
7	CODD
7	DARWIN

Fourth Normal Form (4NF)

- 4NF requires that there be no non-trivial multivalued dependencies of attribute sets on something other than a superset of a candidate key.
- A table is said to be in 4NF if and only if it is in the BCNF and multivalued dependencies are functional dependencies. The 4NF removes unwanted data structures (redundancy): multivalued dependencies.

...Fourth Normal Form (4NF)

- That is; if $X \rightarrow\rightarrow Y$ is non-trivial multivalued dependency in R, then
  - X is superkey for schema R.

Example:

- Consider relation R and its dependency set
  - R(A, B, C, D) and F = {AB →CD, AB →C}
- Then the relation can be normalized as:
  - R1(A, B, C) and R1(A, B, D)

Fifth Normal Form (5NF or PJNF)

- ***A join dependency (JD)***, denoted by JD(R1, R2, … Rn), specified on relational schema R, specifies a constraint on the state r of R. The constraint states that every legal state r of R have a nonadditive join decomposition into R1, R2, … Rn.
- Join dependency is a general form of multivalued dependency where n = 2. (i.e. JD(R1, R2) implies R1∩R2 →→R1-R2 and

. . .Fifth Normal Form (5NF or PJNF)

- 5NF also known as Project-Join Normal Form (PJNF) requires that there are no non-trivial join dependencies that do not follow from the key constraints. A table is said to be in the 5NF if and only if it is in 4NF and every join dependency in it is implied by the candidate keys.
- That is; if JD(R1, R2, … Rn) is non-trivial join dependency in R, then
  - Every Ri is superkey of R.

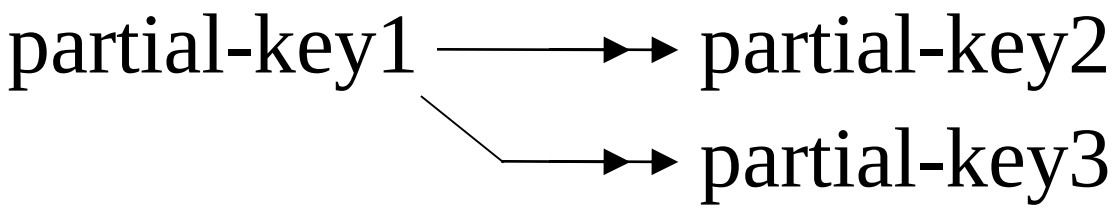
...Fifth Normal Form (5NF or PJNF)

- A join dependency (JD), denoted by JD(R1, R2, … Rn), specified on relational schema R, specifies a constraint on the state r of R. The constraint states that every legal state r of R have a nonadditive join decomposition into R1, R2, … Rn.
- Although, there are also other higher level normalizations such as DKNF and 6NF, most relational database designs are sufficiently normalized at BCNF level or even at 3NF.

4th Normal Form

4th Normal Form is violated if:

- Boyce Codd Normal Form is violated
- If there exists a partial key which has multiple independent multi-valued functional dependencies to other partial keys.

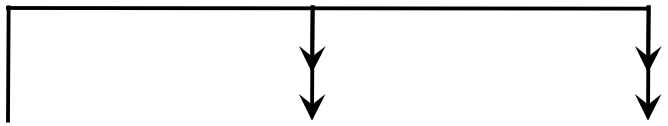


4NF Example – Violation

Instruments_Languages

Name	Instrument	Language
Fred	Piano	French
Fred	Flute	Italian
Fred	Flute	Spanish
Jane	Piano	French
Jane	Oboe	French
Sam	Piano	French
Sam	Oboe	Spanish
Sam	Flute	Spanish

4NF Example – Violation



Name	Instrument	Language
Fred	Piano	French
Fred	Flute	Italian
Fred	Flute	Spanish
Jane	Piano	French
Jane	Oboe	French
Sam	Piano	French
Sam	Oboe	Spanish
Sam	Flute	Spanish

Does this relation violate 1st, 2nd, 3rd, or BCNF?
Are there any redundant facts?

4NF Example – Correction

LanguagesSpoken

Name	Language
Fred	French
Fred	Italian
Fred	Spanish
Jane	French
Sam	French
Sam	Spanish

InstrumentsPlayed

Name	Instrument
Fred	Piano
Fred	Flute
Jane	Piano
Jane	Oboe
Sam	Piano
Sam	Oboe
Sam	Flute

-End-